Day 4 – OOPs concepts

Encapsulation in Python:

Encapsulation is one of the core concepts of Object Oriented Programming (OOP).

- The idea of encapsulation is to bind the data members and methods into a single unit.
- Helps in better maintainability, readability and usability as we do not need to explicitly pass data members to member methods
- Helps maintain data integrity by allowing validation and control over the values assigned to variables.
- It helps in achieving abstraction. A class can hide the implementation part and discloses only the functionalities required by other classes which allows later changes to representations or implementations without impacting the codes that uses this class.

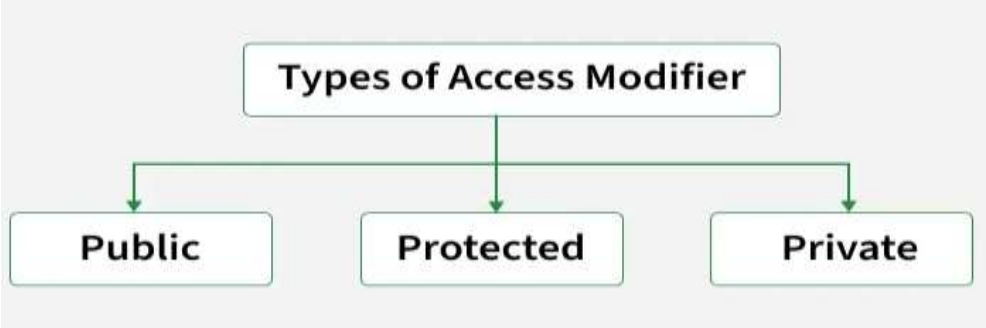
Example:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name          # public attribute
        self.__salary = salary    # private attribute
emp = Employee("Fedrick", 50000)
print(emp.name)
print(emp.__salary)
```

Why do we need Encapsulation?

- Protects data from unauthorized access and accidental modification.
- Controls data updates using getter/setter methods with validation.
- Enhances modularity by hiding internal implementation details.
- Simplifies maintenance through centralized data handling logic.
- Reflects real-world scenarios like restricting direct access to a bank account balance.

Access Specifiers:
Access specifiers define how class members (variables and methods) can be accessed from outside the class. They help in implementing encapsulation by controlling the visibility of data. There are three types of access specifiers:



Declaring Protected and Private Methods

In Python, you can control method access levels using naming conventions:

- Use a single underscore (_) before a method name to indicate it is protected meant to be used within class or its subclasses.
- Use a double underscore (__) to define a private method accessible only within class due to name mangling.

What is Modularity?

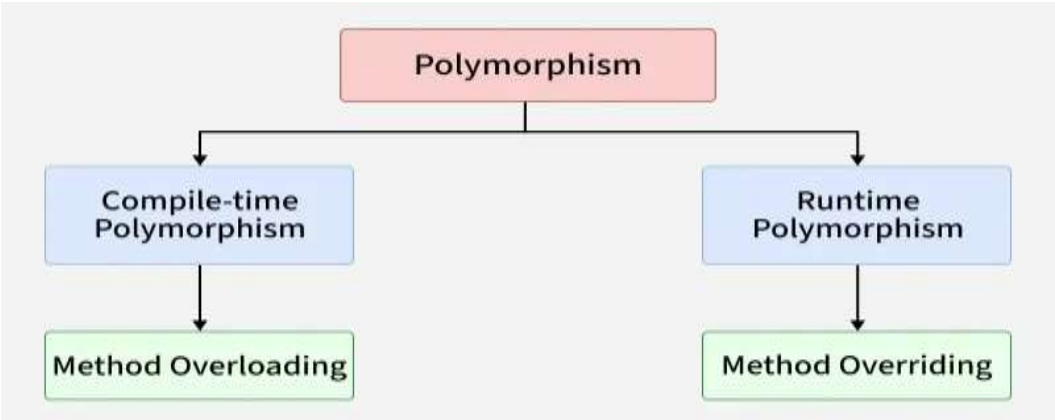
Modularity refers to the concept of making multiple modules first and then linking and combining them to form a complete system (i.e, the extent to which a software/Web application may be divided into smaller modules is called modularity). Modularity enables re-usability and will minimize duplication.

Polymorphism in Python:

Polymorphism means "many forms" and allows the same method, function or operator to behave differently depending on the object or data it works with. This flexibility helps create more reusable, maintainable and scalable code.

Types of Polymorphism

Polymorphism refers to the ability of the same method or operation to behave differently based on object or context. It mainly includes compile-time and runtime polymorphism.



1. Compile-time Polymorphism

Compile-time polymorphism involves selecting a method or operation before program execution, typically through method or operator overloading. Languages such as Java and C++ support this feature.

Python does not support true compile-time polymorphism because method calls are resolved at runtime. However, similar behavior can be achieved using default arguments, variable-length arguments (*args), or keyword arguments (**kwargs).

Example:

```
class Calculator:
    def multiply(self, a=1, b=1, *args):
        result = a * b
        for num in args:
            result *= num
        return result
calc = Calculator()
print(calc.multiply())
print(calc.multiply(4))
print(calc.multiply(2, 3))
print(calc.multiply(2, 3, 4))
```

Output:

```
1
4
6
24
```

2. Runtime Polymorphism (Overriding)

Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it. This happens through Method Overriding a child class provides its own version of a method already defined in the parent class.

Example:

```
class Animal:
    def sound(self):
        return "Some generic sound"
```

```
class Dog(Animal):
    def sound(self):
        return "Bark"
class Cat(Animal):
    def sound(self):
        return "Meow"
animals = [Dog(), Cat(), Animal()]
for animal in animals:
    print(animal.sound())
```

Output:

Bark

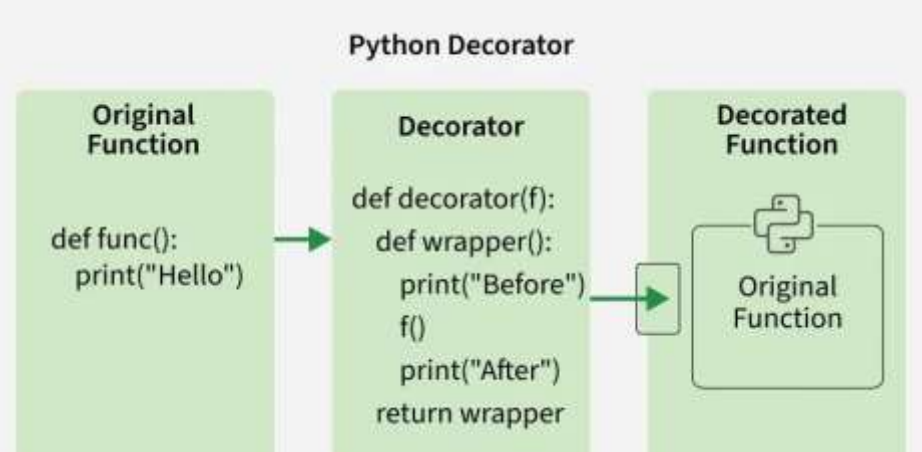
Meow

Some generic sound

Decorators in Python:

Decorators are flexible way to modify or extend behavior of functions or methods, without changing their actual code.

- A decorator is essentially a function that takes another function as an argument and returns a new function with enhanced functionality.
- They are often used in scenarios such as logging, authentication and memoization, allowing us to add additional functionality to existing functions or methods in a clean, reusable way.



Example:

```
def decorator(func):
    def wrapper():
        print("Before calling the function.")
        func()
        print("After calling the function.")
    return wrapper
@decorator
def greet():
    print("Hello, World!")
greet()
```

Output:

```
Before calling the function.
Hello, World!
After calling the function.
```

Built-in Decorators

1. @staticmethod: used to define a method that doesn't operate on an instance of class (i.e., it doesn't use self). Static methods are called on class itself, not on an instance of class.

Example:

```
class MathOperations:
    @staticmethod
    def add(x, y):
        return x + y


res = MathOperations.add(5, 3)
print(res)
```

2. @classmethod: used to define a method that operates on class itself (i.e., it uses cls). Class methods can access and modify class state that applies across all instances of class.

Example:

```
class Employee:
```

```
    raise_amount = 1.05
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    @classmethod
    def set_raise_amount(cls, amount):
        cls.raise_amount = amount
Employee.set_raise_amount(1.10)
print(Employee.raise_amount)
```

Abstraction in Python:

Abstraction is the process of hiding implementation details and exposing only the essential functionality to the user. It is used to hide the implementation details from the user and expose only necessary parts, making the code simpler and easier to interact with.

Abstract Base Class:

Abstract Base Class (ABC) is used to achieve data abstraction by defining a common interface for its subclasses. It cannot be instantiated directly and serves as a blueprint for other classes.

Abstract classes are created using abc module and @abstractmethod decorator, allowing developers to enforce method implementation in subclasses while hiding complex internal logic.

Example:

```
from abc import ABC, abstractmethod
class Greet(ABC):
    @abstractmethod
    def say_hello(self):
        pass  # Abstract method
class English(Greet):
    def say_hello(self):
        return "Hello!"
g = English()
print(g.say_hello())
```

Python Exception Handling:

Exception Handling allows a program to handle unexpected errors during execution in a controlled way, instead of crashing abruptly. It enables programs to detect errors, manage them properly and continue execution wherever possible.

- Handles runtime errors such as invalid input, file not found, division by zero and type mismatches that occur during program execution.
- Helps improve program reliability by ensuring the application does not terminate unexpectedly when an error occurs.

Example:

```
n = 10

try:
    res = n / 0
except ZeroDivisionError:
    print("Can't be divided by zero!")
```

Example:

```
try:
    n = 0
    res = 100 / n
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Enter a valid number!")
else:
    print("Result is", res)
finally:
    print("Execution complete.")
```

Raise an Exception

We raise an exception using the raise keyword followed by an instance of the exception class that we want to trigger. We can choose from built-in exceptions or define our own custom exceptions by inheriting from Python's built-in Exception class.

Syntax:

```
raise ExceptionType("Error message")
```

Example:

```
def set(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    print(f"Age set to {age}")
try:
    set(-5)
except ValueError as e:
    print(e)
```